

第 01 章 — 一次工具调用

TL;DR

--

为什么这很重要

“4,892,769” “”
bug--

--

核心概念

模型负责写请求，你的代码负责干活

--“

”--

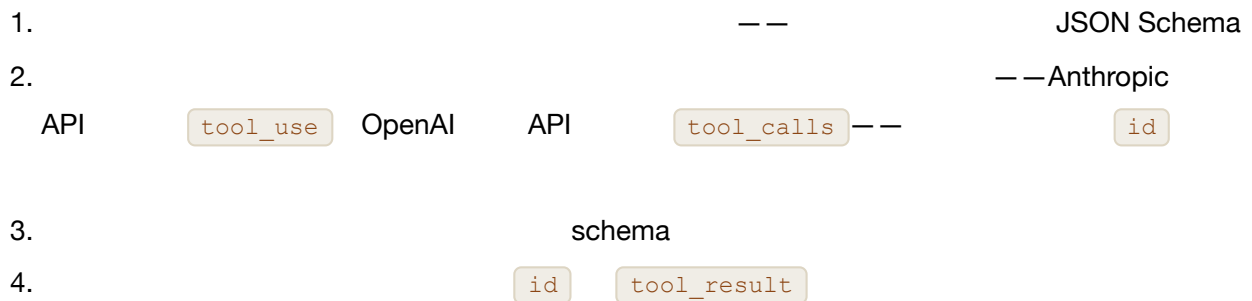
“ ”

--

四步循环

```
sequenceDiagram
    participant App as
    participant LLM as
    participant Tool as

    App->>LLM: + schema
    LLM-->>App: tool_use
    App->>Tool: execute(name, arguments)
    Tool-->>App:
    App->>LLM: tool_result
    LLM-->>App:
```



```
// 2 --
{ id: "call_abc", name: "get_weather", input: { city: "Tokyo" } }
```

```
// 4 -- id content
{ tool_use_id: "call_abc", content: "18°C, partly cloudy" }
```

shell

Schema 就是契约

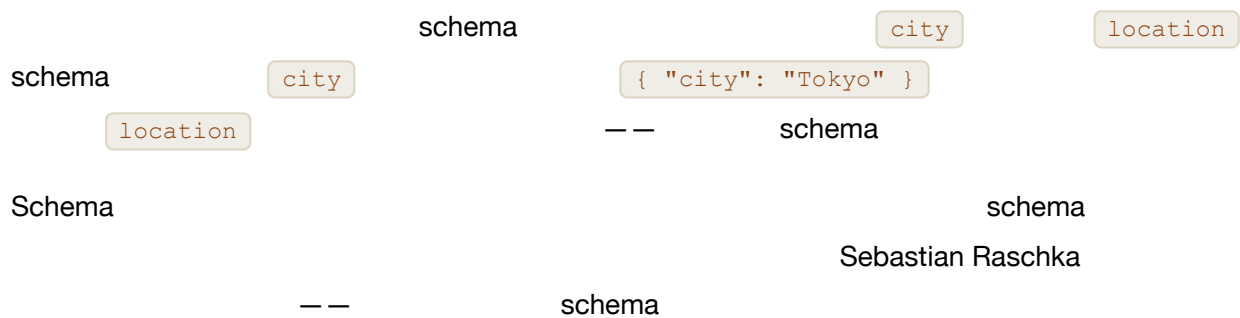
- **Name** --
- **Description** --
“ ”
- **schema** Input schema -- JSON Schema

```
//      --      SDK
{
  name: "get_weather",
  description: "
      ",
  input_schema: {
    type: "object",
    properties: { city: { type: "string", description: "      'Tokyo'" } },
    required: ["city"]
  }
}
```

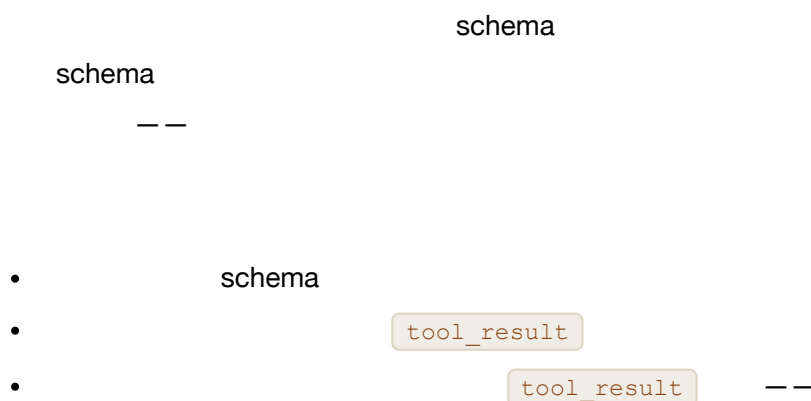
bug

“ X”

Schema 和函数必须同步变化



错误输入和故障应该成为消息，而不是异常



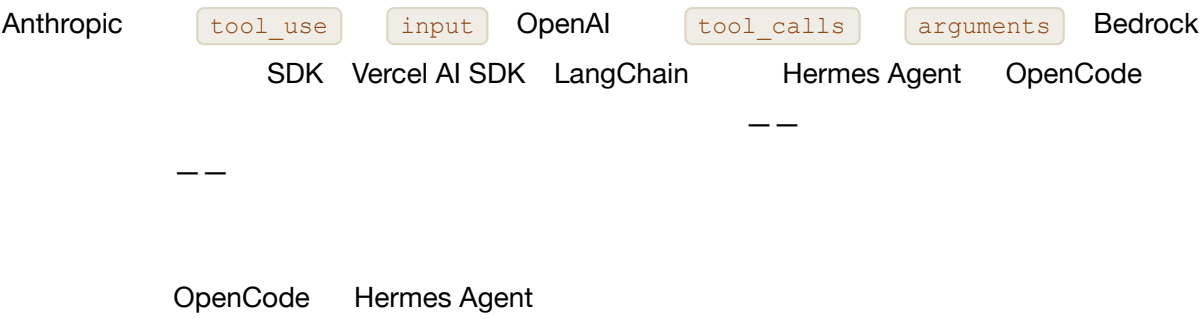
工具结果也有自己的结构



值得了解的服务商特定控制项

- `tool_choice` “X” `none` Anthropic OpenAI Bedrock Gemini
- `tool_use` OpenAI `parallel_tool_calls: false` 02
- `schema` OpenAI `strict: true` JSON Schema schema JSON Schema “ schema JSON” `response_format` — —
- schema — —
- refusal OpenAI Anthropic — —

服务商各不相同，概念始终一致



真实系统笔记

- **OpenCode** `Tool.define` schema “ ”
- **Hermes Agent** `ToolEntry` schema
- **OpenClaw** **Paperclip** “ ” shell “ + schema + ”

常见失败情况

- AI `tool_use` “ `tool_use` ” `tool_result` `id` schema `schema` `tool_result`

•

`tool_choice`

•

`tool_result`

02

•

“ ” _ _

与你的智能体结对

• “

JSON ”

• “

schema

”

• “

`tool_result`

”

• “

4 KB

”

• “

OpenCode

_ _

”

下一步

02